

MongoDB Assessment - Module 5 (M5-A1) - QuickEats

Section A – Concept Application

S1.

Explain why a document-oriented NoSQL database like MongoDB is better suited for this situation than a relational database. Address schema flexibility, data nesting, and horizontal scalability, and describe at least one concrete trade-off of choosing MongoDB.

Answer:

A document database stores each record as a self-contained JSON-like document instead of splitting it across rigid, pre-defined tables, which matters here for three reasons: Schema flexibility: In MySQL, adding a field like `isVeg` or `addOns` to menu items means an `ALTER TABLE` on every affected table plus a migration to backfill old rows. In MongoDB, new documents can simply include the new field going forward - no schema migration needed. Data nesting: A menu item naturally has price and `addOns` nested inside it, and an order naturally embeds its list of items. MongoDB documents nest this directly. MySQL would need separate `order_items` and `addons` tables joined by foreign keys, so reconstructing one order means multiple `JOINS`. Horizontal scalability: MongoDB is built to shard data across many servers by a shard key, so the app scales by adding machines. Scaling MySQL horizontally usually means manual, painful sharding or read-replica setups. Trade-off: Denormalized documents duplicate data - a restaurant's name may appear inside thousands of order documents, so renaming a restaurant means updating many documents (or accepting temporary staleness), instead of one `UPDATE` in a normalized relational schema. MongoDB also lacks the mature multi-table `JOIN` and strict multi-document transaction guarantees that relational databases offer by default.

```
// Example: nested document vs. relational tables
{
  _id: ObjectId("..."),
  name: "Punjabi Tadka",
  cuisine: "Indian",
  menu: [
    { itemName: "Butter Chicken", price: 320, isVeg: false, addOns: ["Extra Gravy"] },
    { itemName: "Dal Makhani", price: 220, isVeg: true, addOns: [] }
  ]
}
// One document holds the whole menu - no JOIN needed, and adding a new
// field like preparationTime to any item requires no schema change.
```

S2.

Describe the difference between `insertOne` and `insertMany`. Write the mongosh commands to create a database `quickeats_db`, create a collection `orders`, and insert two orders - one with `insertOne` and one batch of three orders with `insertMany`. Explain why `insertMany` is preferred over multiple `insertOne` calls for bulk inserts.

Answer:

`insertOne()` inserts a single document and returns one `insertedId`. `insertMany()` takes an array of documents and inserts all of them in one call, returning an array of `insertedIds`. `insertMany` is preferred for bulk inserts because it sends one batched request to the server instead of `N` separate round trips, which cuts down network latency overhead significantly and lets the server process the batch more efficiently.

internally.

```
use quickeats_db

db.createCollection("orders")

// insertOne - a single order
db.orders.insertOne({
  customerName: "Rohan Mehta",
  items: [ { name: "Paneer Tikka", quantity: 1, price: 250 } ],
  totalAmount: 250,
  deliveryAddress: "12 MG Road, Pune",
  status: "pending"
})

// insertMany - a batch of three orders
db.orders.insertMany([
  {
    customerName: "Sneha Patil",
    items: [ { name: "Veg Biryani", quantity: 2, price: 180 } ],
    totalAmount: 360,
    deliveryAddress: "22 FC Road, Pune",
    status: "confirmed"
  },
  {
    customerName: "Aman Verma",
    items: [ { name: "Chicken Roll", quantity: 3, price: 90 } ],
    totalAmount: 270,
    deliveryAddress: "5 Park Street, Kolkata",
    status: "delivered"
  },
  {
    customerName: "Priya Nair",
    items: [ { name: "Masala Dosa", quantity: 1, price: 120 } ],
    totalAmount: 120,
    deliveryAddress: "9 MG Road, Bengaluru",
    status: "pending"
  }
])
```

S3.

Write a MongoDB query using \$in, \$eq, and \$gt to retrieve only matching orders. Also write a query that uses projection to return only customerName, totalAmount, and status, with results sorted by totalAmount descending and limited to 10 documents. Explain what each operator does in this context.

Answer:

\$eq matches documents where a field is exactly equal to a value (the same as writing field: value directly). \$gt matches documents where a numeric/date field is strictly greater than a given value. \$in matches documents where a field's value is any one of the values in a given array - useful here since we want 'pending' OR 'confirmed', not a single fixed status.

```
// Query 1: status in ['pending','confirmed'], specific customer, totalAmount > 500
db.orders.find({
  status: { $in: ["pending", "confirmed"] },
  customerName: { $eq: "Rohan Mehta" },
  totalAmount: { $gt: 500 }
})

// Query 2: projection + sort + limit
db.orders.find(
  { status: { $in: ["pending", "confirmed"] } },
  { customerName: 1, totalAmount: 1, status: 1, _id: 0 }
```

```
).sort({ totalAmount: -1 }).limit(10)
```

S4.

Explain the difference between \$set, \$push, and \$addToSet. Then write the updateOne command that sets status to 'confirmed' and restaurantNote to 'Order accepted', and pushes {event: 'confirmed', time: new Date()} into the statusHistory array. Justify which operator you chose for each field.

Answer:

\$set replaces (or creates) the value of a single field without touching any other fields in the document - it works on any field type, not just arrays. \$push appends a new element onto the end of an array field, growing its length every time, and allows duplicate values. \$addToSet also appends to an array, but only if an identical element isn't already present, effectively treating the array like a mathematical set. Justification: status and restaurantNote are plain scalar fields being overwritten, so \$set is correct for both. statusHistory is a running log of events, and each entry carries its own timestamp (new Date()), so entries are never truly identical - using \$push is correct because we want every event recorded, including ones that might look similar; \$addToSet would provide no real de-duplication benefit here since the time field makes each object unique anyway, and its intent (uniqueness) doesn't match a history log's intent (completeness).

```
db.orders.updateOne(
  { _id: ObjectId("PUT_ORDER_ID_HERE") },
  {
    $set: { status: "confirmed", restaurantNote: "Order accepted" },
    $push: { statusHistory: { event: "confirmed", time: new Date() } }
  }
)
```

S5.

Write a MongoDB aggregation pipeline using \$match, \$group, \$project, and \$sort to produce a report of total revenue per restaurant, number of completed orders per restaurant, ranked by total revenue. Explain what each stage does and why aggregation pipelines are more efficient for this task than reading all documents into Node.js and calculating totals there.

Answer:

\$match filters the collection early (like a WHERE clause), so only 'delivered' orders flow into the rest of the pipeline. \$group buckets the remaining documents by restaurantName and computes totalRevenue (sum of totalAmount) and completedOrders (a count) per bucket - similar to SQL's GROUP BY with aggregate functions. \$project reshapes the output, renaming _id back to restaurantName and dropping fields we don't need. \$sort orders the final grouped results by totalRevenue descending. This is more efficient than pulling every document into Node.js because the computation happens inside the database engine, right next to the data, using MongoDB's optimized internal execution (and any available indexes on the \$match fields) instead of serializing potentially hundreds of thousands of documents over the network, holding them all in application memory, and looping over them in JavaScript. Aggregation avoids that network and memory bottleneck entirely.

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $group: {
    _id: "$restaurantName",
    totalRevenue: { $sum: "$totalAmount" },
    completedOrders: { $sum: 1 }
  } },
  { $project: {
    _id: 0,
    restaurantName: "$_id",
```

```

        totalRevenue: 1,
        completedOrders: 1
    } },
    { $sort: { totalRevenue: -1 } }
])

```

S6.

Explain how indexes work in MongoDB and why they reduce read query time. Write the command to create a compound index on {customerId: 1, status: 1}. Then evaluate the trade-off: under what conditions does adding an index hurt performance, and under what conditions is it clearly worth the cost?

Answer:

An index in MongoDB is a separate, ordered data structure (a B-tree) that stores a sorted reference to each document based on the indexed field's value(s). Without an index, a query must perform a 'collection scan', checking every single document one by one. With an index, the query planner can binary-search the ordered B-tree and jump almost directly to matching documents, turning an $O(n)$ scan into something closer to $O(\log n)$ - which is why indexed lookups on a 500,000-document collection can drop from seconds to milliseconds. Trade-off: Every insert, update, or delete that touches an indexed field must also update that field's index entry, adding write overhead, and each index consumes extra disk space and RAM (for the index working set). Adding an index hurts when the collection is small (a full scan is already fast), the indexed field has low selectivity (e.g., a boolean, which narrows results very little), or the workload is write-heavy with very few reads on that field. It's clearly worth the cost here because customerId + status is a highly selective compound key (it narrows results down to a specific customer's specific-status orders), the collection is large (500,000 documents), and this exact filter combination is used constantly by the order search feature - the read-time savings far outweigh the small write and storage overhead.

```
db.orders.createIndex({ customerId: 1, status: 1 })
```

Section B – Practical Coding Tasks

Task 1: Set Up the QuickEats Restaurant Database

Create quickeats_db and a restaurants collection. Insert at least 5 restaurant documents using insertMany (name, cuisine, city, rating, isOpen, menu array). Use find() with no filter to display all inserted documents.

Answer:

```
use quickeats_db
```

```
db.createCollection("restaurants")
```

```

db.restaurants.insertMany([
  { name: "Punjabi Tadka", cuisine: "Indian", city: "Mumbai", rating: 4.5, isOpen: true,
    menu: [ { itemName: "Butter Chicken", price: 320 }, { itemName: "Dal Makhani", price: 220 } ] },
  { name: "Mainland China", cuisine: "Chinese", city: "Pune", rating: 4.2, isOpen: true,
    menu: [ { itemName: "Hakka Noodles", price: 250 }, { itemName: "Manchurian", price: 210 } ] },
  { name: "Pizza Express", cuisine: "Italian", city: "Bengaluru", rating: 4.0, isOpen: false,
    menu: [ { itemName: "Margherita Pizza", price: 299 }, { itemName: "Pasta Alfredo", price: 260 } ] },
  { name: "Taco Bell", cuisine: "Mexican", city: "Delhi", rating: 3.8, isOpen: true,
    menu: [ { itemName: "Crunchy Taco", price: 99 }, { itemName: "Burrito Bowl", price: 199 } ] },
  { name: "Saffron House", cuisine: "Indian", city: "Mumbai", rating: 4.7, isOpen: true,
    menu: [ { itemName: "Chicken Biryani", price: 280 }, { itemName: "Paneer Tikka", price: 240 } ] }
])

```

```
// Verify
```

```
db.restaurants.find()
```

Task 2: Query and Filter Orders Using Operators

Insert at least 8 order documents (customerId, restaurantName, items, totalAmount, status, orderDate).
Query with \$gt + \$eq, then a separate \$in query, then projection with sort and limit.

Answer:

```
db.orders.insertMany([
  { customerId: "CUST001", restaurantName: "Punjabi Tadka", items: [{name:"Butter Chicken", qty:1, price:32}], totalAmount: 32, status: "pending", orderDate: new Date() },
  { customerId: "CUST002", restaurantName: "Mainland China", items: [{name:"Hakka Noodles", qty:2, price:25}], totalAmount: 50, status: "confirmed", orderDate: new Date() },
  { customerId: "CUST003", restaurantName: "Pizza Express", items: [{name:"Margherita Pizza", qty:1, price:199}], totalAmount: 199, status: "confirmed", orderDate: new Date() },
  { customerId: "CUST001", restaurantName: "Taco Bell", items: [{name:"Burrito Bowl", qty:2, price:199}], totalAmount: 398, status: "pending", orderDate: new Date() },
  { customerId: "CUST004", restaurantName: "Saffron House", items: [{name:"Chicken Biryani", qty:2, price:220}], totalAmount: 440, status: "confirmed", orderDate: new Date() },
  { customerId: "CUST002", restaurantName: "Punjabi Tadka", items: [{name:"Dal Makhani", qty:1, price:220}], totalAmount: 220, status: "confirmed", orderDate: new Date() },
  { customerId: "CUST005", restaurantName: "Mainland China", items: [{name:"Manchurian", qty:3, price:210}], totalAmount: 630, status: "confirmed", orderDate: new Date() },
  { customerId: "CUST003", restaurantName: "Saffron House", items: [{name:"Paneer Tikka", qty:2, price:240}], totalAmount: 480, status: "confirmed", orderDate: new Date() }
])

// Query using $gt and $eq
db.orders.find({ totalAmount: { $gt: 400 }, status: { $eq: "delivered" } })

// Separate query using $in
db.orders.find({ status: { $in: ["pending", "confirmed"] } })

// Projection + sort + limit
db.orders.find(
  {},
  { customerId: 1, totalAmount: 1, status: 1, _id: 0 }
).sort({ totalAmount: -1 }).limit(5)
```

Task 3: Update Order Records Using Update Operators

Use updateOne with \$set to confirm a pending order and add confirmedAt. Use updateMany with \$set to mark all confirmed orders as delivered. Use \$push to add a deliveryLog entry. Use \$unset to remove confirmedAt from at least one order. Verify with find() after each update.

Answer:

```
// 1. updateOne + $set: confirm one pending order
db.orders.updateOne(
  { status: "pending" },
  { $set: { status: "confirmed", confirmedAt: new Date() } }
)
db.orders.find({ status: "confirmed" })

// 2. updateMany + $set: batch-dispatch all confirmed orders to delivered
db.orders.updateMany(
  { status: "confirmed" },
  { $set: { status: "delivered" } }
)
db.orders.find({ status: "delivered" })

// 3. $push: add a dispatch event to one specific order's deliveryLog array
db.orders.updateOne(
  { customerId: "CUST001" },
  { $push: { deliveryLog: { event: "dispatched", time: new Date() } } }
)
db.orders.find({ customerId: "CUST001" })

// 4. $unset: remove confirmedAt from at least one order
db.orders.updateOne(
  { customerId: "CUST001" },
  { $unset: { confirmedAt: "" } }
)
db.orders.find({ customerId: "CUST001" })
```

Task 4: Restaurant Revenue Report Using Aggregation

Build a multi-stage aggregation pipeline: \$match delivered orders, \$group by restaurantName (totalRevenue, orderCount), \$project clean output suppressing _id, \$sort by totalRevenue descending, \$limit to top 3.

Answer:

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $group: {
    _id: "$restaurantName",
    totalRevenue: { $sum: "$totalAmount" },
    orderCount: { $sum: 1 }
  } },
  { $project: {
    _id: 0,
    restaurantName: "$_id",
    totalRevenue: 1,
    orderCount: 1
  } },
  { $sort: { totalRevenue: -1 } },
  { $limit: 3 }
])
```

Section C – Mini Capstone Project

QuickEats Food Delivery Backend Database

Build quickeats_db with three collections (restaurants, orders, customers), each with 5+ realistic nested documents. Include at least 3 read queries (\$gt/\$lt/\$in/\$eq + projection), 3 update operations (\$set/\$push/\$inc), a revenue aggregation pipeline, and a compound index with an explain('executionStats') comparison.

Answer:

use quickeats_db

```
// ----- Collections & sample documents -----
// restaurants (reuse Task 1's 5 documents) and orders (reuse Task 2's 8 documents) already exist.

db.customers.insertMany([
  { customerId: "CUST001", name: "Rohan Mehta", city: "Pune", loyaltyPoints: 120,
    addresses: [ { label: "Home", city: "Pune", street: "12 MG Road" } ] },
  { customerId: "CUST002", name: "Sneha Patil", city: "Mumbai", loyaltyPoints: 80,
    addresses: [ { label: "Home", city: "Mumbai", street: "22 FC Road" } ] },
  { customerId: "CUST003", name: "Aman Verma", city: "Kolkata", loyaltyPoints: 45,
    addresses: [ { label: "Office", city: "Kolkata", street: "5 Park Street" } ] },
  { customerId: "CUST004", name: "Priya Nair", city: "Bengaluru", loyaltyPoints: 200,
    addresses: [ { label: "Home", city: "Bengaluru", street: "9 MG Road" } ] },
  { customerId: "CUST005", name: "Karan Shah", city: "Delhi", loyaltyPoints: 30,
    addresses: [ { label: "Home", city: "Delhi", street: "18 CP Block" } ] }
])

// ----- 3 read queries -----
// (1) restaurants with rating above 4
db.restaurants.find({ rating: { $gt: 4 } })

// (2) orders filtered by status using $in
db.orders.find({ status: { $in: ["pending", "confirmed"] } })
```

```
// (3) customers from a specific city, with projection
db.customers.find(
  { city: { $eq: "Mumbai" } },
  { name: 1, city: 1, _id: 0 }
)

// ----- 3 update operations -----
// (1) $set: update an order's status
db.orders.updateOne({ customerId: "CUST002" }, { $set: { status: "delivered" } })

// (2) $push: append to a delivery log array
db.orders.updateOne(
  { customerId: "CUST002" },
  { $push: { deliveryLog: { event: "out_for_delivery", time: new Date() } } }
)

// (3) $inc: increment a customer's loyaltyPoints
db.customers.updateOne({ customerId: "CUST001" }, { $inc: { loyaltyPoints: 50 } })

// ----- Revenue aggregation pipeline -----
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $group: { _id: "$restaurantName", totalRevenue: { $sum: "$totalAmount" }, orderCount: { $sum: 1 } } },
  { $project: { _id: 0, restaurantName: "$_id", totalRevenue: 1, orderCount: 1 } },
  { $sort: { totalRevenue: -1 } }
])

// ----- Compound index + explain comparison -----
// Run BEFORE creating the index:
db.orders.find({ customerId: "CUST001", status: "delivered" }).explain("executionStats")
// Note totalDocsExamined - without an index this equals (or is close to) the
// total document count, since MongoDB must scan the whole collection.

db.orders.createIndex({ customerId: 1, status: 1 })

// Run AFTER creating the index (same query):
db.orders.find({ customerId: "CUST001", status: "delivered" }).explain("executionStats")
// totalDocsExamined should now drop to (approximately) just the matching documents,
// since the index lets MongoDB jump directly to the relevant entries instead of
// scanning every document in the collection.
```

Section D – AI-Augmented Learning

1. Exact prompt given to the AI tool

The prompt used to generate the initial aggregation pipeline:

Answer:

"Write a MongoDB aggregation pipeline for a collection called deliveries with fields driverId (string), orderId (string), restaurantName (string), deliveryTimeMinutes (number), tip (number), and status (string). Filter for only completed deliveries using \$match, then group by driverId to compute avgDeliveryTime and totalTips per driver. Sort the result by avgDeliveryTime ascending so the fastest drivers appear first. Use \$project to rename output fields as driverID, avgDeliveryTime, and totalTips, suppressing the default _id."

2a. AI's original pipeline

The first version produced by the AI tool:

Answer:

```

db.deliveries.aggregate([
  { $match: { status: "completed" } },
  { $group: {
    _id: "$driverId",
    avgDeliveryTime: { $avg: "$deliveryTimeMinutes" },
    totalTips: { $sum: "$tip" }
  } },
  { $sort: { avgDeliveryTime: 1 } },
  { $project: {
    _id: 0,
    driverID: "$_id",
    avgDeliveryTime: 1,
    totalTips: 1
  } }
])

```

2b. Corrected version (after testing without AI)

After inserting real sample delivery documents and running the pipeline in mongosh, two issues surfaced: avgDeliveryTime came back as long, unrounded decimals (e.g. 23.66666666666667), and there was no way to tell how many deliveries backed each average, so a driver with a single lucky fast delivery could outrank a driver with 50 consistently solid ones. The corrected pipeline rounds the average and adds a deliveryCount field:

Answer:

```

db.deliveries.aggregate([
  { $match: { status: "completed" } },
  { $group: {
    _id: "$driverId",
    avgDeliveryTime: { $avg: "$deliveryTimeMinutes" },
    totalTips: { $sum: "$tip" },
    deliveryCount: { $sum: 1 }
  } },
  { $sort: { avgDeliveryTime: 1 } },
  { $project: {
    _id: 0,
    driverID: "$_id",
    avgDeliveryTime: { $round: ["$avgDeliveryTime", 1] },
    totalTips: 1,
    deliveryCount: 1
  } }
])

```

3. Note on what was changed and why

Summary of the fix:

Answer:

The AI's pipeline was functionally correct and matched every requirement in the prompt, but testing it against real sample data revealed two practical gaps. First, avgDeliveryTime printed as an ugly, unrounded float, which isn't fit for a report a human will read - I added \$round to display one decimal place. Second, and more importantly, the ranking by avgDeliveryTime alone can be misleading: a driver with only one or two completed deliveries can post an artificially fast average and rank above a driver who has completed dozens of deliveries reliably. Adding a deliveryCount field (via \$sum: 1 in the same \$group stage) lets anyone reading the report judge how statistically meaningful each driver's average actually is, without changing the pipeline's overall structure or the fields the assessment originally asked for.